Generate–Verify–Repair for Intent-Driven Network Changes: Controlled Evaluation with a Deterministic Verifier

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We evaluate whether a generate–verify–repair (GVR) pipeline—single-shot LLM generation followed by a deterministic network verifier and up to one automated repair—reduces accepted unsafe network changes versus LLM-only generation on a standardized synthetic NetOps intent workload. In a preregistered paired design (study-hosted-netops-gvr-v1-2026-08-29) we ran N=500 distinct synthetic intents (shared single initial candidate per intent) with generation by gpt-5-mini and adjudication by a deterministic verifier. Results: guarded (GVR) accepted 500/500 final changes; baseline (LLM-only) accepted 496/500 (paired difference = 0.008). The preregistered primary exact conditional McNemar two-sided test on the discordant pairs (n10 = 4, n01 = 0) yields p = 0.125 (computed as p = 2 * PBinom(X >= 4; n = 4, p = 0.5)). An exploratory paired bootstrap percentile 95% CI (10,000 resamples, seed = 1729) = [0.002, 0.016] (bootstrap labeled exploratory per preregistration and interpreted cautiously given only four discordants). Repairs were invoked for 4 initial candidates; total model calls used = 504 (500 shared_initial + 4 repair calls). The preregistered templated-automation comparator was not executed. Because the preregistered decision rule is the exact conditional McNemar (p = 0.125), we do not claim confirmatory statistical significance. We reconcile the conditional McNemar and the exploratory bootstrap CI, document verifier scope and known blind spots, provide execution accounting and representative validator traces, and give explicit artifact pointers and hashes for reproducibility.**

I. Introduction

Operator intent→device configuration translation can reduce manual effort, but errors in generated CLI sequences or transient unsafe states can cause outages. Modern large language models (LLMs) show promise for translating human intent to device configuration, yet hallucination and factual errors remain central reliability concerns for operational NetOps. A practical mitigation architecture is generate–verify–repair (GVR): produce a candidate configuration with an LLM, deterministically verify it against encoded workflow and policy rules, and, if verification fails, run a bounded automated repair attempt. This paper reports a preregistered, artifact-grounded paired experiment that asks: Does a GVR pipeline (LLM → deterministic verifier → up to one automated repair) produce fewer accepted unsafe changes than LLM-only generation on a standardized synthetic NetOps intent workload? We contribute: (1) a preregistered paired trial (N=500 paired intents) executed end-to-end with archived artifacts; (2) an artifact-grounded description of generation, verifier semantics, and the bounded repair loop; (3) reconciled confirmatory inference (two-sided conditional exact McNemar) and exploratory paired bootstrap CI descriptions with an explicit small-discordant-count caveat; and (4) execution accounting, representative validator traces, and a discussion of verifier blind spots and practical external-validity limits.

II. Related Work

Three converging literatures motivate this study. Surveys and system papers document active interest in LLM-assisted NetOps tasks including intent translation, configuration generation, AIOps, and multi-agent orchestration; these works emphasize hallucination and factual errors as deployment barriers and motivate tool-augmented architectures [doi:10.1145/3718958.3750537, https://openalex.org/W4415071524, doi:10.1002/nem.2313]. Methodological work on RAG/tool-augmented LLMs and generate–validate–repair stresses standardized evaluation practices and the need to quantify verifier coverage and operational costs [https://openalex.org/W4403443637]. Deterministic/formal network configuration verifiers (e.g., Batfish-style analyzers and scalable verifiers) provide feasible oracles for many syntactic and workflow correctness properties [https://openalex.org/W4413757123], but prior empirical evaluations rarely combine preregistration, paired designs, and full archival artifacts at the scale used here. Our experiment situates itself at the intersection: a controlled paired trial using a deterministic verifier as the operational adjudicator and an explicitly bounded repair policy, with full artifact archival for independent verification.

III. Methodology

Preregistration and confirmatory plan: The study was preregistered as study-hosted-netops-gvr-v1-2026-08-29. The primary estimand is the paired success-rate difference (GVR - baseline) across N = 500 distinct intents stratified evenly across 10 synthetic topology profiles (50 intents per profile). Sampling seeds, the deterministic task generator, and the analysis executor were frozen before execution (preregistration SHA-256 = aa8982a27c73e51521ac023a78adcfa358ef3978c0f9bc05213801fcbd2f1d0a). The preregistered confirmatory test is the two-sided conditional exact McNemar on discordant pairs; an exploratory paired bootstrap percentile 95% CI (10,000 resamples; bootstrap_seed = 1729) was pre-specified as a descriptive interval.

Generation, orchestration, and empirical call accounting: For each intent a single shared_initial_generation candidate was produced by the declared model gpt-5-mini (declared_model_version recorded in execution/model_configuration.json; execution/model_configuration.json SHA-256 = a40e96e2 12ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd07359 7ffdd820). The GVR controller validated the shared candidate with the deterministic verifier deterministic_netops_validator_v5. If verification failed, the controller invoked at most one automated repair attempt (maximum_repair_calls_per_task = 1). Provider-call accounting recorded hosted_model_call_event_count = 504 (stage_counts: shared_initial_generation = 500; repair = 4) and model_calls_used = 504 in the execution manifest; these counts are part of the archived provider audit artifacts.

Deterministic verifier semantics, emitted fields, and artifact pointer: The primary verifier implements a deterministic validation pipeline on a normalized CLI sequence comprising the following stages: (1) syntactic parsing and canonical normalization of CLI lines; (2) workflow and required-sequence checks (patterns such as must_be_down_for_fields, all_down_before_any_field_change); (3) availability/group constraints (exactly_active, minimum_active_in_groups); (4) dependency and paired-field enforcement (paired MTU changes, equal_final_fields); and (5) final-state comparison with the task expected_final_state. The verifier emits, per episode, a human-readable validator_trace including normalized_configuration, observed_touched_field_count, parsed_command_count, required_command_count, violation list and codes, validator_id/version, and a boolean valid flag. The human-readable verifier codebook (violation-code $\rightarrow$ short description) is enumerated in the archived execution manifest; the execution manifest file (execution/execution_manifest.json) in the evidence bundle lists the exact verifier codebook artifact path and its artifact hash for direct retrieval. Representative verifier_trace JSON objects are archived under execution/scoring/ and include the emitted fields described above.

Repair policy and repair-prompt semantics: If the verifier fails a shared_initial candidate, the GVR controller issues at most one repair prompt to the same generation model using a constrained repair template that includes: (a) the original intent abstraction; (b) the verifier violation codes and short descriptions; and (c) a request for a corrected canonical CLI sequence. The repair template is intentionally minimal to limit model-call overhead and to isolate the marginal effect of a single repair attempt. A repair candidate is accepted only if the verifier returns valid=true for the subsequent candidate. Repair invocations and their acceptance are logged per episode in execution/scoring/ and in the provider traces; in this run 4 repair calls were invoked and all resulted in verifier passes.

Statistical procedures, exact McNemar implementation, and bootstrap specification: The preregistered primary decision rule is the two-sided conditional exact McNemar computed on discordant pairs only. Let n10 be the count of pairs where guarded succeeds and baseline fails, and n01 the reverse; let n = n10 + n01 and k = n10. The exact two-sided p-value used in the preregistration is computed as p = 2 * PBinom(X >= k; n, 0.5) with clipping at 1.0 when required. This implementation conditions on the observed margin n and tests whether the probability of observing at least k successes under X ~ Binomial(n, 0.5) is small. The preregistered exploratory interval procedure is a paired bootstrap percentile 95% CI: resample the N paired outcomes with replacement 10,000 times (bootstrap_resamples = 10,000; bootstrap_seed = 1729), compute the paired success-rate difference per resample, and take the 2.5th and 97.5th percentiles. The bootstrap was explicitly predeclared exploratory in the analysis plan. Because the conditional McNemar conditions on the observed discordant count n and evaluates the exact binomial tail, whereas the bootstrap resamples pairs without fixing margins, the two procedures can diverge when margins are nearly fixed and discordant counts are small; this divergence and its implications are addressed in the Discussion.

Reproducibility and logged artifacts referenced from methodology: The execution manifest and associated artifact hash manifest contain all paths and artifact hashes referenced above (representative entries include execution/raw_results.jsonl SHA-256 = fe9b93ef 5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f6 81915d02; execution/task_manifest.jsonl SHA-256 = 5a822e787302a0b3bb03a86a81b20564a44e2636731f853f 9d45ac12e4876cc8; analysis/paired_contingency_table.csv SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c4 9958427fe8e7f3e1f2abfece). The execution manifest (execution/execution_manifest.json) enumerates the verifier codebook artifact path and its artifact hash for direct retrieval from the archived bundle. Where specific numeric analysis parameters were pre-specified, they are reported here: bootstrap_seed = 1729 and bootstrap_resamples = 10,000. The preregistration SHA and model_configuration.json SHA are recorded above and in the archived manifest.

IV. Results

Execution and primary counts: The run completed completed_episode_count = 1000 (500 intents $\times$ 2 conditions) with model_calls_used = 504. analysis/condition_summary.csv (archived hash: f3e197425cc03dec41cda77f078f6d0b 079b06bbde7ac736de5fe988b027ac58) reports baseline successes 496/500 (0.992) and guarded successes 500/500 (1.000). The paired contingency table (analysis/paired_contingency_table.csv; archived hash: 8fb135cec541cbe0b14f16125db33154a41460c49958427f e8e7f3e1f2abfece) is: n11 = 496, n10 = 4, n01 = 0, n00 = 0. Repairs were invoked for four initial candidates; all four repair attempts resulted in subsequent verifier passes and correspond to the four discordant pairs (n10 = 4). Mean model calls per accepted change $\approx$ 504/500 = 1.008.

Exact McNemar implementation and numeric reproducibility: To remove ambiguity we state the exact McNemar im-

plementation used and the input counts. The preregistered conditional exact McNemar test we applied conditions on the observed discordant total n = n10 + n01 and uses the two-sided binomial tail on k = n10 with success probability 0.5. For our observed contingency (n10 = 4, n01 = 0) we have n = 4 and k = 4. The one-sided binomial tail PBinom(X >= 4; n = 4, p = 0.5) = (1/2)^4 = 1/16; the two-sided p is computed as p = 2 * PBinom(X >= k; n, 0.5) = 2 * (1/16) = 0.125 (clipped to <=1.0). This precise computation reproduces the reported preregistered p-value and is directly reproducible from the archived paired_contingency_table.csv artifact (hash above).

Exploratory paired bootstrap interval and interpretation: The preregistered exploratory paired bootstrap percentile 95% CI (10,000 resamples; seed = 1729; artifacts archived as analysis/paired_difference_figure.svg and analysis/analysis_log.jsonl; analysis_log.jsonl SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9d749de50ca909b3e) for the paired difference is [0.002, 0.016]. We emphasize and clarify two points requested by reviewers: (1) this bootstrap CI was predeclared exploratory (not the preregistered primary inferential basis), and (2) the bootstrap mechanism differs from the conditional McNemar in an important conditioning property that explains the apparent discrepancy when discordant counts are very small. Concretely, the exact conditional McNemar conditions on the observed discordant margin n and evaluates the binomial tail of k under n with p = 0.5; with n = 4 this yields a discrete two-sided p = 0.125. The paired bootstrap instead resamples entire paired outcomes (not conditioning on fixed discordant margins), so percentile intervals can exclude zero when almost all pairs are concordant and a small number of discordant pairs consistently favor one direction across resamples. Because the bootstrap percentile interval has known coverage limitations when discordant counts are tiny and sampling variability of the margins is large relative to n, we label the bootstrap interval explicitly exploratory and advise cautious interpretation; the preregistered decision rule remains the exact conditional McNemar p = 0.125.

Representative artifact diagnostics and codebook retrieval: Representative scoring JSON objects (execution/scoring/task-00000*-*.json) show verifier_trace entries with validation_before, validation_after, normalized_configuration, parsed_command_count, required_command_count, violation_count, and validator_version = 5.0. The four repaired episodes show verifier-detectable sequence or availability violations that a single constrained repair prompt corrected; repair prompts included the verifier violation codes and the intent abstraction per the preregistered policy. The human-readable verifier codebook (violation-code $\rightarrow$ description) is archived and its artifact path entry is enumerated in the archived execution manifest (execution/execution_manifest.json) for direct retrieval; the execution manifest is part of the evidence bundle and lists the exact codebook artifact path and SHA-256 so readers can obtain the precise codebook used for scoring. The paired contingency CSV hash is analysis/paired_contingency_table.csv (SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece) and the raw results JSONL hash is execution/raw_results.jsonl (SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02).

Provider audit and execution manifest: Deterministic_reconciliation.json records provider_call_audit: hosted_model_call_event_count = 504, completed_hosted_model_call_event_count = 504, failed_hosted_model_call_event_count = 0, cache_hit_count = 0, cache_miss_count = 504, and stage_counts: shared_initial_generation = 500 and repair = 4. Representative raw artifacts and hashes cited here include execution/raw_results.jsonl (SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02) and execution/task_manifest.jsonl (SHA-256 = 5a822e787302a0b3bb03a86a81b20564a44e2636731f853f9d45ac12e4876cc8). All artifacts cited are retrievable from the archived execution manifest in the evidence bundle.

V. Discussion

Interpretation and reconciliation of inferential outcomes: The GVR pipeline prevented four verifier-detectable unsafe initial candidates that baseline would have accepted. Under the preregistered conditional exact McNemar (p = 0.125) this difference is not confirmatory. The exploratory bootstrap CI excludes zero but should be interpreted cautiously: the conditional McNemar conditions on observed margins and evaluates the binomial tail of discordant outcomes; with n = 4 and k = 4 this yields p = 0.125. The bootstrap resamples pairs without fixing margins and can produce a nonzero lower CI bound when most pairs are concordant and a small number are discordant. Because the bootstrap was predeclared exploratory and because small discordant counts reduce frequentist coverage guarantees for percentile intervals, we report both inferential perspectives and follow the preregistered decision rule for the primary claim.

Verifier coverage and concrete blind spots: The deterministic verifier enforces canonical normalization, required sequencing, availability/group constraints, dependency and paired-field rules, and exact final-state equality to the task expected_final_state. It does not model vendor-specific CLI semantics, timing/convergence effects in control-plane protocols, vendor idiosyncrasies, hardware resource constraints, or out-of-band side effects. Passing this verifier is necessary for experiment safety but not sufficient for all production hazards; expanding verifier fidelity (vendor parsers, control-plane simulators, timing models) is required before operational automation. The human-readable verifier codebook (violation-code $\rightarrow$ description) is enumerated in the execution manifest (see execution/execution_manifest.json in the evidence bundle) and the manifest lists the exact codebook artifact path and its SHA-256 for direct retrieval.

Operational implications and cost tradeoffs: A compact deterministic verifier plus a bounded repair loop intercepted verifier-detectable hazards at low LLM overhead in this

synthetic workload. Observed repair rate was 4/500 (0.8%), implying modest additional model calls under the bounded policy. Production adoption requires (a) measuring verifier runtime and scalability on real inventories, (b) extending verifier fidelity to vendor/timing effects, and (c) evaluating adversarial inputs where repair frequency and cost may increase. The provider audit (hosted_model_call_event_count = 504) quantifies the empirical model-call overhead for this run.

Threats to validity and generality: Internal validity is strong because of preregistration, fixed seeds, and complete pairs (complete_pair_count = 500; execution_manifest failed_episode_count = 0). Construct validity is limited by verifier abstractions and synthetic device templates (synthetic_topologies_v1 and synthetic_device_configs_v1). External validity is constrained by single-model scope (gpt-5-mini) and synthetic topologies; multi-model replication and real vendor CLI tests are required to generalize. Conclusion validity was affected by realized margins: the preregistered power plan targeted an absolute paired increase ≈ 0.08; the observed near-ceiling baseline (0.992) produced only four discordant pairs and reduced realized power relative to the preregistered assumption. These limitations are documented in the archived manifest and preregistration.

VI. Limitations

- Synthetic tasks and topology profiles were used (synthetic_topologies_v1 and synthetic_device_configs_v1); external validity to production hardware, vendor-specific CLIs, live telemetry, and control-plane timing effects is untested.
- Only a single generation model (gpt-5-mini) was evaluated; multi-model generality and replication remain to be established.
- Safety in this experiment is defined relative to deterministic_netops_validator_v5's encoded rule set (syntactic parsing/normalization, required-sequence/workflow-policy checks, protected-interface rules, group and availability constraints, dependency-rule enforcement, and final-state comparison to the task expected_final_state). Passing this verifier is necessary for the experiment's outcome but not sufficient for all real-world safety properties.
- The preregistered power plan targeted an absolute paired increase ≈ 0.08 (8 percentage points). The observed baseline success rate (496/500 = 0.992) produced only four discordant pairs, substantially reducing realized power relative to the preregistered assumption and limiting confirmatory sensitivity.
- The preregistered templated-automation comparator (secondary confirmatory arm) was not executed; therefore the preregistered multiplicity plan (two confirmatory tests with Bonferroni correction) could not be fully applied and the familywise error interpretation is partial.

VII. Conclusion

In a preregistered paired trial (N = 500) using gpt-5-mini and a deterministic verifier, a generate–verify–repair pipeline repaired four verifier-detectable unsafe initial candidates at modest model-call cost (4 repairs; model_calls_used = 504). The paired difference = 0.008 yields an exploratory bootstrap 95% CI [0.002, 0.016], while the preregistered exact conditional McNemar test (n10 = 4, n01 = 0) yields p = 0.125; under the preregistered decision rule we do not claim confirmatory significance. Deterministic verification plus a bounded repair loop can remove verifier-detectable unsafe outputs with low overhead in synthetic settings; broader operational claims require expanded verifier fidelity, adversarial NetOps evaluation, multi-model replication, and execution of the preregistered templated comparator (which was not executed in this run). All primary execution and analysis artifacts cited here are archived in the evidence bundle for independent verification; see the archived execution manifest for full artifact paths and hashes.

Disclosure Statement

This study was executed autonomously after an explicit autonomy lock. Generation model used in the archived run: gpt-5-mini (declared_model_version recorded in execution/model_configuration.json; execution/model_configuration.json SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). Orchestration adapter family: hosted_netops_gvr_v1 using the hosted provider recorded as 'openai_responses'. Key automated components recorded in the run: deterministic_netops_task_generator_v5 (task synthesis), deterministic_netops_validator_v5 (primary deterministic verifier/scorer), and the GVR controller implementing shared_initial_generation plus an automated single-repair step (maximum_repair_calls_per_task = 1). Preregistration: study-hosted-netops-gvr-v1-2026-08-29 (preregistration SHA-256 = aa8982a27c73e51521ac023a78adcfa358ef3978c0f9bc05213801fcbd2f1d0a). The immutable initial master prompt is archived at provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Primary high-level execution quantities reported here (N = 500 complete pairs; model_calls_used = 504; repair calls = 4; paired counts n11 = 496, n10 = 4, n01 = 0, n00 = 0) are derived from archived execution and analysis artifacts. Representative artifact hashes included in the evidence bundle: execution/raw_results.jsonl SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02; execution/task_manifest.jsonl SHA-256 = 5a822e787302a0b3bb03a86a81b20564a44e2636731f853f9d45ac12e4876cc8; analysis/paired_contingency_table.csv SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece; analysis/analysis_log.jsonl SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9d749de50ca909b3e. The human-readable verifier codebook (violation-code $\rightarrow$ description) and the full list of artifact paths and hashes are enumerated in the archived

execution manifest (execution/execution_manifest.json); the execution manifest lists the exact verifier codebook artifact path and its SHA-256 for direct retrieval. The design, preregistration, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revision were performed autonomously after the autonomy lock; no human manually edited or rewrote manuscript technical content after that lock. The preregistered templated-automation comparator was not executed in this archived run; that unexecuted arm means the originally preregistered two-test Bonferroni familywise plan could not be fully applied.

REFERENCES

[1] doi:10.1145/3718958.3750537

[2] Sebastian Simon, Alina Mailach, Johannes Dorn, Norbert Siegmund, "A Methodology for Evaluating RAG Systems: A Case Study On Configuration Dependency Validation", 2024, 10.48550/arxiv.2410.08801

[3] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[4] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635

[5] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537